

Module 18 Structured Notes: Doubly Linked List Foundations

1. Current Progress Analysis

You are now moving from **circular singly linked lists** to **doubly linked lists**.

Before this module, you learned many important linked-list ideas:

Previous idea	Why it matters in Module 18
Arrays vs linked lists	You understand why linked lists use nodes instead of fixed-size array blocks.
Node structure	You know that a node stores data and links.
Pointer movement	You know that <code>p = p->next</code> moves a pointer forward.
<code>first</code> / <code>Head</code> pointer	You know the list needs an outside pointer to reach the first node.
Traversal	You know how to visit nodes one by one.
Display	You know how to print all node values.
Length/count	You know how to count nodes by traversal.
<code>first</code> , <code>last</code> , and <code>t</code>	You know these pointers are useful when creating a list from an array.
Circular singly linked lists	You know that not every list ends at <code>NULL</code> ; circular lists stop when they return to <code>Head</code> .

Module 18 introduces a new linked-list structure:

Doubly linked list

The main difference is this:

Singly linked-list node:
[data | next]

Doubly linked-list node:
[prev | data | next]

A singly linked list can move only forward.

A doubly linked list can move forward and backward.

That is the main idea of Module 18.

2. Big Idea of Module 18

A **doubly linked list** is a linked list where every node has two links:

1. one link to the previous node,
2. one link to the next node.

Each node stores three things:

```
previous address + data + next address
```

Simple node picture:

```
[ prev | data | next ]
```

Example list:

```
NULL <- [10] <-> [20] <-> [30] -> NULL
```

This means:

```
10 has no previous node.  
10 points forward to 20.  
  
20 points backward to 10.  
20 points forward to 30.  
  
30 points backward to 20.  
30 has no next node.
```

So a doubly linked list gives us movement in two directions.

Forward movement:

```
p = p->next;
```

Backward movement:

```
p = p->prev;
```

3. Why It Is Called “Doubly” Linked

The word **doubly** means each node has two links.

Link	Meaning
prev	points to the previous node
next	points to the next node

A singly linked list has only one direction:

```
10 -> 20 -> 30 -> NULL
```

A doubly linked list has two directions:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
```

So the list can be read like this:

Forward:

```
10 -> 20 -> 30
```

Backward:

```
30 -> 20 -> 10
```

4. Doubly Linked List Node Structure

In C, a doubly linked-list node is written like this:

```
struct Node {
    struct Node *prev;
    int data;
    struct Node *next;
};
```

Meaning of each field:

Field	Meaning
prev	stores the address of the previous node
data	stores the actual value
next	stores the address of the next node

Example:

[prev | data | next]

If the node stores 20, it may look like this:

[address of 10 | 20 | address of 30]

That means:

20's previous node is 10.
20's next node is 30.

5. Important Correction: prev and next Must Be Pointers

This is correct:

```
struct Node {
    struct Node *prev;
    int data;
    struct Node *next;
};
```

This is wrong:

```
struct Node {  
    struct Node prev;    // wrong  
    int data;  
    struct Node next;    // wrong  
};
```

Why is it wrong?

Because this would mean:

A Node contains a full Node.
That Node contains another full Node.
That Node contains another full Node.
And so on forever.

The compiler cannot create a structure with infinite size.

So we use pointers:

```
struct Node *prev;  
struct Node *next;
```

A pointer stores only an address.

That makes the node a fixed size:

one previous pointer + one integer + one next pointer

6. Linear Doubly Linked List Shape

Module 18 is about a **linear doubly linked list**.

A linear doubly linked list has a clear beginning and a clear end.

Example:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
```

The first node has:

```
prev = NULL
```

The last node has:

```
next = NULL
```

This is important.

A normal doubly linked list is **not circular**.

It does not loop back to the first node.

7. Meaning of NULL in a Doubly Linked List

NULL means the pointer does not point to a valid node.

In a doubly linked list, NULL appears in two important places.

First node

The first node has no previous node.

So:

```
first->prev = NULL;
```

Picture:

```
NULL <- [10]
```

Last node

The last node has no next node.

So:

```
last->next = NULL;
```

Picture:

```
[30] -> NULL
```

Full example:

```
NULL <- [10] <-> [20] <-> [30] -> NULL
```

8. One-Node Doubly Linked List

Suppose we create one node containing 10.

Picture:

```
first
|
v
NULL <- [10] -> NULL
```

For a one-node doubly linked list:

```
first->prev = NULL;
first->data = 10;
first->next = NULL;
```

Why is prev equal to NULL ?

Because there is no node before 10.

Why is next equal to NULL ?

Because there is no node after 10.

So one node is both:

```
first node
last node
```

9. The Role of `first`

A linked list needs a pointer to the first node.

For Module 18, this pointer is called:

```
first
```

Example:

```
first
 |
 v
NULL <- 10 <-> 20 <-> 30 -> NULL
```

`first` stores the address of the first node.

If `first` is lost, the whole list becomes unreachable.

So the beginner rule still applies:

```
Do not move first carelessly.
Use a temporary pointer like p for traversal.
```

Correct traversal idea:

```
struct Node *p = first;
```

Then move `p`:

```
p = p->next;
```

Do not move `first` unless you are intentionally changing the beginning of the list.

10. Pointers Used During Creation

To create a doubly linked list from an array, we use three important pointers.

Pointer	Meaning
<code>first</code>	points to the first node of the list
<code>last</code>	points to the current last node while building the list
<code>t</code>	temporary pointer for the new node being created

These are similar to singly linked-list creation.

The difference is that in a doubly linked list, every new node must connect in two directions:

```
previous link + next link
```

11. Creating a Doubly Linked List From an Array

Suppose we have this array:

```
int A[] = {10, 20, 30};
```

We want to create this doubly linked list:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
```

The creation process is:

1. Create the first node separately.
2. Set the first node's `prev` to `NULL`.
3. Set the first node's `next` to `NULL`.
4. Make `last` point to the first node.
5. For each remaining array value, create a new node `t`.
6. Set `t->prev = last`.
7. Set `t->next = NULL`.
8. Connect the old last node to the new node using `last->next = t`.
9. Move `last` to `t`.

The key extra line compared with singly linked list creation is:

```
t->prev = last;
```

This creates the backward link.

12. Full C Program for Module 18

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    struct Node *prev;
    int data;
    struct Node *next;
};

struct Node *first = NULL;

void create(int A[], int n) {
    struct Node *t;
    struct Node *last;

    if (n <= 0) {
        first = NULL;
        return;
    }

    // Create first node
    first = (struct Node *)malloc(sizeof(struct Node));
    first->prev = NULL;
    first->data = A[0];
    first->next = NULL;

    last = first;

    // Create remaining nodes
    for (int i = 1; i < n; i++) {
        t = (struct Node *)malloc(sizeof(struct Node));

        t->prev = last;
        t->data = A[i];
        t->next = NULL;

        last->next = t;
        last = t;
    }
}

void display(struct Node *p) {
    while (p != NULL) {
```

```

        printf("%d ", p->data);
        p = p->next;
    }
}

int length(struct Node *p) {
    int len = 0;

    while (p != NULL) {
        len++;
        p = p->next;
    }

    return len;
}

int main() {
    int A[] = {10, 20, 30};

    create(A, 3);

    printf("Doubly linked list: ");
    display(first);

    printf("\nLength: %d\n", length(first));

    return 0;
}

```

Expected output:

```

Doubly linked list: 10 20 30
Length: 3

```

13. Explanation of the Node Definition

Code:

```

struct Node {
    struct Node *prev;
    int data;
}

```

```
struct Node *next;  
};
```

```
struct Node *prev;
```

This stores the address of the previous node.

Example:

```
10 <-> 20
```

For node 20 :

```
20's prev points to 10
```

```
int data;
```

This stores the actual value.

Example:

```
data = 20
```

```
struct Node *next;
```

This stores the address of the next node.

Example:

```
20 <-> 30
```

For node 20 :

```
20's next points to 30
```

14. Explanation of the Global Pointer

Code:

```
struct Node *first = NULL;
```

Meaning:

first is a pointer to a Node.
At the beginning, the list is empty.
So first points to NULL.

Picture:

```
first -> NULL
```

After creating a list:

```
first -> 10 <-> 20 <-> 30 -> NULL
```

15. Explanation of `create()`

Function header:

```
void create(int A[], int n)
```

Meaning:

Parameter	Meaning
<code>A[]</code>	array containing values to put into the list
<code>n</code>	number of values in the array

Example call:

```
int A[] = {10, 20, 30};  
create(A, 3);
```

This should create:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
```

16. Empty Array Check

Code:

```
if (n <= 0) {  
    first = NULL;  
    return;  
}
```

Meaning:

If the array has no values, we cannot create any node.

So the list should be empty:

```
first -> NULL
```

Then we return from the function.

This prevents errors such as trying to access:

```
A[0]
```

when the array has no valid first element.

17. Creating the First Node

Code:

```
first = (struct Node *)malloc(sizeof(struct Node));  
first->prev = NULL;  
first->data = A[0];  
first->next = NULL;  
  
last = first;
```

Meaning:

Step 1

Create a new node in heap memory:

```
first = (struct Node *)malloc(sizeof(struct Node));
```

Now `first` points to the new node.

Step 2

Set the previous link:

```
first->prev = NULL;
```

The first node has no previous node.

Step 3

Store the first array value:

```
first->data = A[0];
```

If `A[0]` is `10`, the node stores `10`.

Step 4

Set the next link:

```
first->next = NULL;
```

At this moment, there is no next node.

Step 5

Make `last` point to the first node:

```
last = first;
```

Because there is only one node, it is both the first and last node.

Picture:

```
first
 |
 v
NULL <- [10] -> NULL
      ^
      |
      last
```

18. Creating the Remaining Nodes

Code:

```
for (int i = 1; i < n; i++) {
    t = (struct Node *)malloc(sizeof(struct Node));

    t->prev = last;
    t->data = A[i];
    t->next = NULL;

    last->next = t;
    last = t;
}
```

This loop creates all nodes after the first node.

For every new value:

1. Create a new node `t`.
2. Make `t->prev` point to the current last node.
3. Put the array value in `t->data`.
4. Make `t->next` equal to `NULL` because it will become the new last node.
5. Make the old last node point forward to `t`.
6. Move `last` to `t`.

19. The Most Important Creation Links

When adding a new node after `last`, these lines are very important:


```
t->prev = last;  
last->next = t;  
last = t;
```

Line 1

```
t->prev = last;
```

The new node points backward to the old last node.

Line 2

```
last->next = t;
```

The old last node points forward to the new node.

Line 3

```
last = t;
```

`last` moves to the new final node.

Together, they create this connection:

```
old last <-> new node
```

20. Dry Run of `create(A, 3)`

Array:

```
A = {10, 20, 30}
```

Goal:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
```

Step 1: Create First Node With 10

Code:

```
first->prev = NULL;  
first->data = 10;  
first->next = NULL;  
last = first;
```

Picture:

```
first  
  |  
  v  
NULL <- [10] -> NULL  
        ^  
        |  
      last
```

Meaning:

```
10 has no previous node.  
10 has no next node yet.  
first points to 10.  
last points to 10.
```

Step 2: Add 20

Create new node t:

```
[ ? | 20 | ? ]
```

Set its links:

```
t->prev = last;  
t->data = 20;  
t->next = NULL;
```

Now:

```
t's prev points to 10.  
t stores 20.  
t's next is NULL.
```

Attach it:

```
last->next = t;  
last = t;
```

Picture:

```
NULL <- [10] <-> [20] -> NULL  
                ^  
                |  
              last
```

Meaning:

```
10->next points to 20.  
20->prev points to 10.  
20->next is NULL.  
last points to 20.
```

Step 3: Add 30

Create new node `t`:

```
[ ? | 30 | ? ]
```

Set its links:

```
t->prev = last;  
t->data = 30;  
t->next = NULL;
```

Before attaching, `last` points to `20`.

So:

```
30->prev points to 20.
```

Attach it:

```
last->next = t;  
last = t;
```

Final picture:

```
NULL <- [10] <-> [20] <-> [30] -> NULL  
                               ^  
                               |  
                             last
```

Final list:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
```

21. Display Function

Code:

```
void display(struct Node *p) {  
    while (p != NULL) {  
        printf("%d ", p->data);  
        p = p->next;  
    }  
}
```

Display means printing all node values.

For this list:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
```

Expected output:

```
10 20 30
```

22. Explanation of `display()`

Function header:

```
void display(struct Node *p)
```

The function receives a pointer to the first node.

Usually we call it like this:

```
display(first);
```

Inside the function, `p` is a copy of `first`.

So moving `p` does not move `first`.

Loop:

```
while (p != NULL)
```

This means:

```
Keep printing while p points to a real node.  
Stop when p becomes NULL.
```

Print current node:

```
printf("%d ", p->data);
```

Move forward:

```
p = p->next;
```

Even though this is a doubly linked list, forward display still uses `next`.

23. Dry Run of `display(first)`

List:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
```

Call:

```
display(first);
```

Dry run:

Step	<code>p</code> points to	Action	Output so far
Start	10	begin	empty
1	10	print 10, move to 20	10
2	20	print 20, move to 30	10 20
3	30	print 30, move to NULL	10 20 30
4	NULL	stop	10 20 30

Final output:

```
10 20 30
```

24. Length Function

Code:

```
int length(struct Node *p) {
    int len = 0;

    while (p != NULL) {
        len++;
        p = p->next;
    }
}
```

```
    return len;  
}
```

Length means counting how many nodes are in the list.

Example:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
```

There are three nodes.

So the length is:

```
3
```

25. Explanation of `length()`

Function header:

```
int length(struct Node *p)
```

The function receives a pointer to the first node.

Counter:

```
int len = 0;
```

At the start, we have counted zero nodes.

Loop:

```
while (p != NULL)
```

Keep counting while `p` points to a real node.

Count the current node:

```
len++;
```

Move forward:

```
p = p->next;
```

Return final count:

```
return len;
```

26. Dry Run of `length(first)`

List:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
```

Call:

```
length(first)
```

Dry run:

Step	<code>p</code> points to	Action	<code>len</code>
Start	10	begin	0
1	10	count 10, move to 20	1
2	20	count 20, move to 30	2
3	30	count 30, move to NULL	3
4	NULL	stop	3

Final answer:

```
3
```

27. Why Display and Length Still Use `next`

A doubly linked list has both `prev` and `next`.

But if we want to move from the first node to the last node, we use:

```
p = p->next;
```

That is forward movement.

So display and length still look similar to singly linked list versions.

The difference is not in the traversal code.

The difference is in the node structure.

Singly node:

```
[data | next]
```

Doubly node:

```
[prev | data | next]
```

28. Forward and Backward Movement

A doubly linked list allows two types of movement.

Forward movement

```
p = p->next;
```

Example:

```
10 -> 20 -> 30
```

Backward movement

```
p = p->prev;
```

Example:

```
30 -> 20 -> 10
```

This is the main advantage of a doubly linked list.

In a singly linked list, if you are at node `30`, you cannot directly go back to `20`.

In a doubly linked list, you can.

29. Singly Linked List vs Doubly Linked List

Feature	Singly linked list	Doubly linked list
Node shape	<code>[data next]</code>	<code>[prev data next]</code>
Forward movement	Yes	Yes
Backward movement	No	Yes
First node	has no previous pointer field	<code>prev</code> is <code>NULL</code>
Last node	<code>next</code> is <code>NULL</code>	<code>next</code> is <code>NULL</code>
Memory use	less memory	more memory
Link changes	fewer links	more links
Navigation	one direction	two directions

Simple comparison:

Singly linked list:
Less memory, but only forward movement.

Doubly linked list:
More memory, but forward and backward movement.

30. Doubly Linked List vs Circular Singly Linked List

This is important because you just finished circular singly linked lists.

Feature	Circular singly linked list	Linear doubly linked list
Node fields	<code>data</code> , <code>next</code>	<code>prev</code> , <code>data</code> , <code>next</code>
Last node points to	<code>Head</code>	<code>NULL</code>
Can move backward directly?	No	Yes
Traversal stop condition	return to <code>Head</code>	reach <code>NULL</code>
Shape	loop	line

Circular singly list:

```
10 -> 20 -> 30
^         |
|         |
|         |
```

Linear doubly list:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
```

So do not confuse these two structures.

Module 18 is not circular.

Module 18 is about a normal linear doubly linked list.

31. Complexity Notes

Create

Creating a list with `n` nodes takes:

```
O(n) time
```

Why?

Because we create each node once.

It also uses:

$O(n)$ space

Why?

Because we create n nodes in heap memory.

Display

Displaying the list takes:

$O(n)$ time

Why?

Because every node must be visited and printed.

Extra space:

$O(1)$

Why?

Because we only use one temporary pointer p .

Length

Finding the length takes:

$O(n)$ time

Why?

Because every node must be counted.

Extra space:

$O(1)$

Why?

Because we only use a counter and one pointer.

32. Memory Cost of a Doubly Linked List

A singly linked-list node stores:

data + next

A doubly linked-list node stores:

prev + data + next

So a doubly linked list uses more memory per node.

The extra memory is used for the `prev` pointer.

This extra pointer gives us backward movement.

Tradeoff:

More memory cost, but easier two-way movement.

33. Common Beginner Mistake 1: Forgetting `prev`

Wrong creation idea:

```
t->data = A[i];
t->next = NULL;
last->next = t;
last = t;
```

This creates only forward links.

It produces something closer to a singly linked list.

Correct version:

```
t->prev = last;  
t->data = A[i];  
t->next = NULL;  
last->next = t;  
last = t;
```

The line:

```
t->prev = last;
```

is what creates the backward link.

34. Common Beginner Mistake 2: Thinking It Is Circular

A normal doubly linked list is not circular.

Correct Module 18 shape:

```
NULL <- 10 <-> 20 <-> 30 -> NULL
```

Wrong assumption:

```
10 <-> 20 <-> 30  
^           |  
|_____||
```

That second structure is a circular doubly linked list, which is a later topic.

In Module 18, the first node's `prev` is `NULL` and the last node's `next` is `NULL`.

35. Common Beginner Mistake 3: Using Circular Stopping Logic

For circular lists, you used logic like:

```
while (p != Head)
```

But for Module 18, the list is linear.

So display and length should stop at `NULL` :

```
while (p != NULL)
```

Correct for Module 18:

```
while (p != NULL) {  
    p = p->next;  
}
```

36. Common Beginner Mistake 4: Moving `first` During Traversal

Wrong idea:

```
while (first != NULL) {  
    printf("%d ", first->data);  
    first = first->next;  
}
```

Why is this dangerous?

Because `first` moves away from the beginning of the list.

At the end, `first` becomes `NULL` .

Then the list becomes unreachable.

Correct idea:

```
struct Node *p = first;  
  
while (p != NULL) {  
    printf("%d ", p->data);
```

```
p = p->next;  
}
```

Move `p`, not `first`.

37. Common Beginner Mistake 5: Not Setting the First Node's `prev`

Wrong:

```
first->data = A[0];  
first->next = NULL;
```

Missing:

```
first->prev = NULL;
```

Correct:

```
first->prev = NULL;  
first->data = A[0];  
first->next = NULL;
```

The first node must clearly show that there is no previous node.

38. Common Beginner Mistake 6: Not Setting the Last Node's `next`

Every new node created at the end should initially have:

```
t->next = NULL;
```

Why?

Because the new node becomes the last node.

The last node should not point to random memory.

The last node should point to `NULL`.

39. Important Invariants

An invariant is a rule that should stay true while the list is valid.

For a normal doubly linked list:

Rule 1

The first node's previous pointer is `NULL`.

```
first->prev == NULL
```

Rule 2

The last node's next pointer is `NULL`.

```
last->next == NULL
```

Rule 3

For neighboring nodes, links should agree.

If `A->next` points to `B`, then `B->prev` should point back to `A`.

Picture:

```
A <-> B
```

Code relationship:

```
A->next == B  
B->prev == A
```

These rules help you check whether your doubly linked list is correctly connected.

40. Full Concept Summary

Module 18 teaches the foundation of doubly linked lists.

The main ideas are:

1. A doubly linked-list node has three fields: `prev`, `data`, and `next`.
 2. `prev` points to the previous node.
 3. `next` points to the next node.
 4. The first node's `prev` is `NULL`.
 5. The last node's `next` is `NULL`.
 6. A doubly linked list can move forward and backward.
 7. Forward movement uses `p = p->next`.
 8. Backward movement uses `p = p->prev`.
 9. Creating a doubly linked list from an array uses `first`, `last`, and `t`.
 10. The new important creation line is `t->prev = last`.
 11. Display still moves forward using `next`.
 12. Length still counts nodes by moving forward until `NULL`.
 13. A normal doubly linked list is linear, not circular.
 14. Doubly linked lists use more memory than singly linked lists because each node has an extra pointer.
 15. The extra memory allows easier two-way movement.
-

41. Module 18 Mastery Questions

You understand Module 18 if you can answer these questions.

1. What are the three fields of a doubly linked-list node?
 2. What does `prev` store?
 3. What does `next` store?
 4. Why is the first node's `prev` equal to `NULL`?
 5. Why is the last node's `next` equal to `NULL`?
 6. What is the difference between a singly linked list and a doubly linked list?
 7. What is the advantage of a doubly linked list?
 8. What is the disadvantage of a doubly linked list?
 9. Why do we write `t->prev = last` during creation?
 10. Why do we write `last->next = t` during creation?
 11. Why do we move `last = t` after attaching a new node?
 12. Why should display use a temporary pointer `p` instead of moving `first`?
 13. Why does display stop at `NULL` in Module 18?
 14. Why does length stop at `NULL` in Module 18?
 15. Is Module 18's doubly linked list circular or linear?
-

42. Practice Task

Create this array:

```
int A[] = {5, 10, 15, 20};
```

Dry-run the `create()` function by hand.

Draw this final list:

```
NULL <- 5 <-> 10 <-> 15 <-> 20 -> NULL
```

For each node, write:

```
prev points to: ____  
data is: ____  
next points to: ____
```

Example for node `10`:

```
prev points to 5  
data is 10  
next points to 15
```

This practice confirms that you understand both forward and backward links.

43. Final Beginner Rule

For Module 18, remember this simple rule:

```
A doubly linked list is like a singly linked list, but every node also remembers  
the node behind it.
```

Singly linked list:

```
I know the next node.
```

Doubly linked list:

```
I know the previous node and the next node.
```

That is the foundation for all later doubly linked-list operations, including insertion, deletion, and reversal.